Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Görseller ve Sprite Temelleri

PyGame ile Oyun Geliştirme – Hafta 5

Dr. Tolga BERBER

Bu Hafta Ne Öğreneceğiz?

1. Görsel Dosyaları
2. Görsel Manipülasyonu
3. Sprite Sınıfı
4. Update Metodu ve Hareket
5. Oyuncu Sprite
6. Sprite Grupları
7. Mini Proje: Uzay Kaçışı v1
8. Özet ve Kapanış

Görsel Dosyaları

Görsel Formatları

Oyun Geliştirmede Yaygın Formatlar

- **PNG** – Kayıpsız, alpha kanalı destekli
- **JPG** – Kayıplı, arka planlar için ideal
- **BMP** – Sıkıştırmasız, büyük dosya

Altın Kural

Sprite görselleri → **PNG** (alpha destekli)
Arka plan görselleri → **JPG** (dosya boyutu küçük)

Alpha Kanalı Nedir?

Her pikselin saydamlık bilgisi:

- 0 = tamamen şeffaf
- 255 = tamamen opak
- Arası = yarı saydam

Sprite'ların dikdörtgen olmayan görünmesini sağlar.

Görsel Yükleme ve Performans

```
# Temel yukleme
gorsel = pygame.image.load("gemi.png")

# Performansli yukleme (ONERILEN)
gorsel = pygame.image.load(
    "gemi.png"
).convert_alpha()

# Alpha gerektirmeyen gorseller
arkaplan = pygame.image.load(
    "bg.jpg"
).convert()
```

Dikkat

`convert()` veya `convert_alpha()` kullanmamak blit hizini **10 kata kadar** düşürebilir!

`convert()` vs `convert_alpha()`

- `convert()` – Ekran formatina dönüştür, alpha kaybolur
- `convert_alpha()` – Alpha kanalini koruyarak dönüştür

Fallback Stratejisi

Dosya bulunamazsa programin çökmesini önle:

- try/except ile yakala
- Renkli dikdörtgen oluştur
- Geliştirme sırasında hayat kurtarır

Görsel Manipülasyonu

Boyut Değiştirme

```
# Piksel tabanlı olcekleme
kucuk = pygame.transform.scale(
    gorsel, (32, 32)
)

# Kaliteli olcekleme (yavas)
kucuk = pygame.transform.smoothscale(
    gorsel, (32, 32)
)

# En-boy oranini koru
orijinal = gorsel.get_size()
oran = orijinal[1] / orijinal[0]
yeni_gen = 100
yeni_yuk = int(yeni_gen * oran)
olcekli = pygame.transform.scale(
    gorsel, (yeni_gen, yeni_yuk)
)
```

`scale()` vs `smoothscale()`

- `scale()` – Hızlı, pikselimsi
- `smoothscale()` – Kaliteli, yavaş
- Küçük sprite'lar → `scale()` yeterli
- UI/büyük görseller → `smoothscale()`

En-Boy Oranı

Oranı korumazsan görsel deforme olur. Hedef boyutu hesapla!

Döndürme ve Çevirme

```
# Dondurme (saat yonunun tersi)
donmus = pygame.transform.rotate(
    gorsel, 45
)
# DIKKAT: Boyut buyur!
# Her seferinde ORIJINALDEN dondur!

# Cevirme
ayna = pygame.transform.flip(
    gorsel, True, False
)
# (True, False) = yatay cevirme
# (False, True) = dikey cevirme
```

Döndürme Notları

- Pozitif açı = saat yönünün tersi
- Döndürmede Surface boyutu büyür
- Orijinal rect'i sakla, merkezi güncelle

Çevirme Kullanımı

Karakter yön değiştirirken:

- Sağa bakıyor → orijinal
- Sola bakıyor → `flip(True, False)`

Döndürmeden çok daha hızlı!

Birikimli Döndürme

Her karede önceki sonucu döndürme! Kalite bozulur. **Orijinal görseli sakla** ve her seferinde ondan döndür.

Sprite Sınıfı

Sprite Nedir?

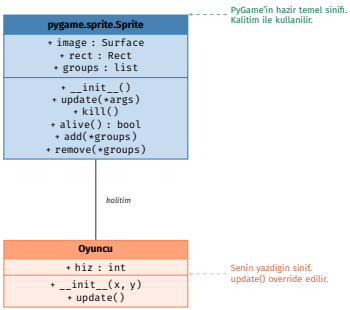
Tanım

Sprite= Görsel + Konum + Davranış

- Oyundaki her bağımsız nesne bir Sprite'tır
- Gemi, düşman, mermi, güç artırıcı...
- OOP ile kapsülleme: her nesne kendi verisini taşır

Neden Sprite Kullanmalıyız?

Ayrı değişkenlerle (gemi_x, gemi_y, dusman1_x...) yönetim kodu karmaşıklaşır. Sprite ile her nesne **kendi kendini yönetir**.



pygame.sprite.Sprite UML yapısı

Sprite Oluşturma – Temel Yapı

```
class KirmiziKare(pygame.sprite.Sprite):
    def __init__(self, x, y):
        # 1. Ust sinifi baslat (ZORUNLU!)
        super().__init__()

        # 2. Gorsel (ZORUNLU!)
        self.image = pygame.Surface((40, 40)
            )
        self.image.fill((220, 30, 30))

        # 3. Konum ve boyut (ZORUNLU!)
        self.rect = self.image.get_rect(
            center=(x, y)
        )

# Kullanim
kare = KirmiziKare(400, 300)
ekran.blit(kare.image, kare.rect)
```

3 Zorunlu Adım

1. `super().__init__()` – Sprite altyapısını başlat
2. `self.image` – Surface nesnesi (görünüm)
3. `self.rect` – Rect nesnesi (konum + boyut)

Eksik Bırakma!

`super().__init__()` unutulursa grup ekleme ve `kill()` çalışmaz. `image`/`rect` eksikse `AttributeError`!

Görsel Dosyası ile Sprite

```python
class UzayGemisi(pygame.sprite.Sprite):
    def __init__(self, x, y):
        super().__init__()
        self.image = gorsel_yukle(
            "assets/images/gemi.png", (50, 40))
        self.rect = self.image.get_rect(
            center=(x, y))
```

```python
# Fallback fonksiyonu
def gorsel_yukle(yol, boyut=None):
    try:
        g = pygame.image.load(yol).convert_alpha()
        if boyut:
            g = pygame.transform.scale(g, boyut)
        return g
    except (pygame.error, FileNotFoundError):
        s = pygame.Surface(boyut or (40, 40))
        s.fill((100, 150, 255))
        return s
```

Kenney Asset Paketi

- kenney.nl – Ücretsiz oyun varlıkları
- Uzay gemileri, düşmanlar, arka planlar
- CC0 lisans – ticari kullanım serbest

gorsel_yukle() Avantajları

- Dosya bulunamazsa çökmez
- Renkli yedek yüzey oluşturur
- Geliştirme sürecini kolaylaştırır

Update Metodu ve Hareket

update() – Sprite'a Hayat Ver

```
class HareketliKare(pygame.sprite.Sprite):
    def __init__(self, x, y):
        super().__init__()
        self.image = pygame.Surface((30, 30)
            )
        self.image.fill((50, 180, 80))
        self.rect = self.image.get_rect(
            center=(x, y)
        )
        self.hiz = 3

    def update(self):
        # Her karede saga kaydir
        self.rect.x += self.hiz

        # Sarmalama: sagdan cik, soldan gir
        if self.rect.left > 800:
            self.rect.right = 0
```

update() Metodu

- Varsayılan olarak **boş**
- Override ederek davranış tanımla
- Hareket, animasyon, kontrol...
- Oyun döngüsünde her karede çağır

Sınır Stratejileri

- **Sarmalama**: Karşıdan geri gir
- **kill()**: Sprite'ı yok et
- **clamp_ip()**: Sınırda tut

kill() – Sprite'ı Yok Etme

```python
class Dusman(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.Surface((40, 40))
        self.image.fill((200, 50, 50))
        self.rect = self.image.get_rect(
            center=(random.randint(40, 760), -20))
        self.hiz = random.uniform(2, 5)

    def update(self):
        self.rect.y += self.hiz
        if self.rect.top > 600:
            self.kill()  # Tum gruplardan sil

# Hayatta mi kontrolu
print(dusman.alive())  # True / False
```

kill() Ne Yapar?

- Sprite'ı tüm gruplardan çıkarır
- Artık güncellenmez ve çizilmez
- Bellekte gereksiz nesne birikmez

alive() Kontrolü

- kill() sonrası → False
- Normal durumda → True

Kullanım Alanları

Mermiler, geçici efektler, ekran dışı nesneler

Oyuncu Sprite

Klavye ile Kontrol Edilen Sprite

```python
class Oyuncu(pygame.sprite.Sprite):
    def __init__(self, x, y):
        super().__init__()
        self.image = gorsel_yukle(
            "assets/images/gemi.png", (50, 40))
        self.rect = self.image.get_rect(
            center=(x, y))
        self.hiz = 5

    def update(self):
        tuslar = pygame.key.get_pressed()
        # Ok tuslari + WASD destegi
        if tuslar[pygame.K_LEFT] or tuslar[pygame.K_a]:
            self.rect.x -= self.hiz
        if tuslar[pygame.K_RIGHT] or tuslar[pygame.K_d]:
            self.rect.x += self.hiz
        if tuslar[pygame.K_UP] or tuslar[pygame.K_w]:
            self.rect.y -= self.hiz
        if tuslar[pygame.K_DOWN] or tuslar[pygame.K_s]:
            self.rect.y += self.hiz
        # Ekran sinirinda tut
        self.rect.clamp_ip(ekran_sinir)
```

Oyuncu Sprite Farkları

- Hareket = kullanıcı girdisi
- Ekran sınırında kalmalı
- `if` (elif degil!) → çapraz hareket

`clamp_ip()`

Rect'i verilen alan içinde tutar. `_ip` = "in place" – oyuncu ekrandan çıkamaz.

`if` vs `elif`

`elif` → sadece bir yön!
`if` → çapraz hareket mümkün.

Sprite Grupları

Group Nedir?

Tanım

Group= Sprite'ları toplu yöneten koleksiyon sınıfı
Bir ordu komutanı gibi: tek komutla herkesi yönetir!

Avantajları

- Toplu çizim: `draw(surface)`
- Toplu güncelleme: `update()`
- Otomatik temizleme: `kill()`
- Çarpışma desteği (Bölüm 6)

İç Yapı

Set (küme) benzeri koleksiyon:

- Tekrar ekleme yok
- O(1) ekleme/çıkarma
- Bir Sprite → N grup

Kural

Gruba yalnızca `Sprite` nesneleri eklenebilir!

Grup İşlemleri

```python
# Grup olustur
tum_spritelar = pygame.sprite.Group()
dusmanlar = pygame.sprite.Group()

# Ekleme
tum_spritelar.add(oyuncu)
tum_spritelar.add(dusman1, dusman2)

# Cikarma
tum_spritelar.remove(dusman1)

# Sorgulama
if tum_spritelar.has(oyuncu):
    print("Aktif")

# Eleman sayisi
print(len(tum_spritelar))

# Tum gruplardan cik (Sprite metodu)
dusman1.kill()
```

Temel Metotlar

`add()`	Gruba ekle
`remove()`	Gruptan çıkar
`has()`	Üyelik kontrolü
`empty()`	Grubu boşalt
`sprites()`	Liste al
`len()`	Eleman sayısı

`kill()` vs `remove()`

`remove()` → sadece o gruptan

`kill()` → **tüm** gruplardan

draw() ve update() – Ana Döngü Sadeleşmesi

Grupsuz (Karmaşık)

```
oyuncu.update()
for d in dusman_listesi:
    d.update()
for y in yildiz_listesi:
    y.update()

ekran.fill(SIYAH)
ekran.blit(oyuncu.image,
           oyuncu.rect)
for d in dusman_listesi:
    ekran.blit(d.image, d.rect)
for y in yildiz_listesi:
    ekran.blit(y.image, y.rect)
```

Gruplu (Sade)

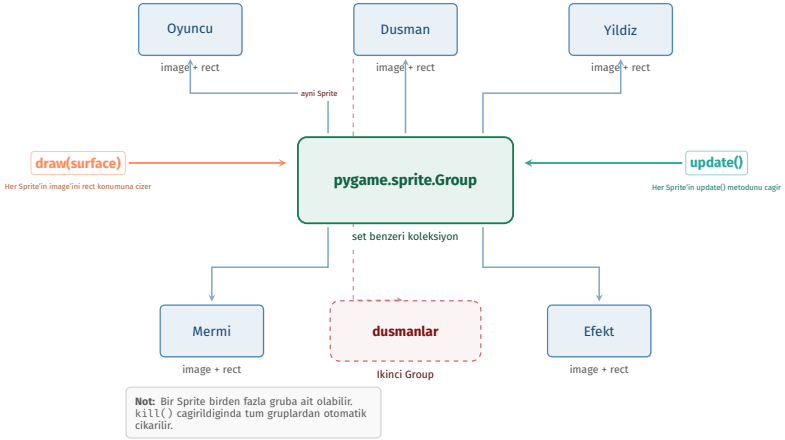
```
# Tek satirda guncelle
tum_spritelar.update()

# Tek satirda ciz
ekran.fill(SIYAH)
tum_spritelar.draw(ekran)



# 2 satir yeterli!
```

Group – Sprite İlişki Diyagramı



Bir Sprite birden fazla gruba ait olabilir. `kill()` tümünden çıkarır.

Mini Proje: Uzay Kaçışı v1

Uzay Kaçışı – Proje Planı

Aşamalı Geliştirme

Versiyon	Bölüm	Özellikler
v1	Bölüm 5	Oyuncu + yıldız arka planı
v2	Bölüm 6	Düşmanlar + mermiler + çarpışma
v3	Bölüm 7	Ses efektleri + müzik
v4	Bölüm 8	Animasyon + patlama + zamanlayıcı

Uzay Kaçışı v1 – Yapı

```python
# Gruplari olustur
tum_spritelar = pygame.sprite.Group()
yildizlar = pygame.sprite.Group()

# 80 yildiz + 1 oyuncu
for _ in range(80):
    y = Yildiz()
    tum_spritelar.add(y)
    yildizlar.add(y)

oyuncu = Oyuncu()
tum_spritelar.add(oyuncu)

# Ana dongu
while calistir:
    # ... olay isleme ...
    tum_spritelar.update()
    ekran.fill(KOYU_MAVI)
    tum_spritelar.draw(ekran)
    pygame.display.flip()
```

2 Grup Stratejisi

- `tum_spritelar` → Çizim
- `yildizlar` → Yıldız yönetimi
- İleride: `dusmanlar`, `mermiler`

81 Nesne, 2 Satır

`update()` → 81 nesneyi güncelle
`draw()` → 81 nesneyi çiz
Gruplar olmasaydı 81 ayrı blit!

Özet ve Kapanış

Bu Hafta Öğrendiklerimiz

Görseller

- PNG vs JPG
- `image.load()`
- `convert_alpha()`
- Alpha kanalı

Manipülasyon

- `scale()` / `smoothscale()`
- `rotate()`
- `flip()`
- En-boy oranı

Sprite Sınıfı

- `self.image`
- `self.rect`
- `super().__init__()`
- `update()` override

Sprite Grupları

- `Group()` oluşturma
- `add()` / `kill()`
- `draw()` / `update()`
- Uzay Kaçışı v1

Gelecek Hafta: Çarpışma Algılama ve Fizik Temelleri – Sprite'lar birbirleriyle etkileşime geçiyor!

Lab Çalışması: Uzay Kaçışı – Başlangıç

Görev

Uzay Kaçışı projesinin temelini oluştur:

1. Oyuncu gemisi Sprite'ı
2. Klavye ile kontrol (WASD + ok)
3. Düşman Sprite'ları (otomatik hareket)
4. Arka plan yıldız efekti

Henüz Yok

Çarpışma algılama ve mermi sistemi gelecek hafta!

Bonus Görevler

- Oyuncuya hız artırma (Shift ile turbo)
- Farklı düşman tipleri (boyut, hız, renk)
- Arka plan kaydırma (parallax efekti)
- Düşman sayacı ekranda göster

Ödev

- Kitaptan Bölüm 5'i okuyun
- Lab egzersizlerini tamamlayın
- Uzay Kaçışı projesinin temelini oluşturun:
  1. Oyuncu gemisi Sprite'ı (WASD + ok tuşları)
  2. En az 5 düşman Sprite'ı (farklı hızlarda)
  3. Ekran sınırı kontrolü (`clamp_ip`)
  4. Yıldız arka plan efekti

Kaynaklar

`pygame.org/docs/ref/sprite.html` – Sprite modülü
`pygame.org/docs/ref/transform.html` – Transform modülü
`kenney.nl/assets` – Ücretsiz oyun varlıkları

Sorular?

Teşekkürler!